kubectl_cheatsheet

kubectl is the command line client for Kubernetes: it talks to the cluster in your current context to inspect, create and change resources. These are the commands you reach for every day, grouped by task. Switch to the k alias and every command updates. Click a command to copy it.

:setup — 4 cmds

Shell completion and the k alias. Add these lines to ~/.bashrc to keep them.

Turn on bash completion

```
$ source <(kubectl completion bash)
```

Turn on zsh completion

```
$ source <(kubectl completion zsh)
```

Use k as an alias, with completion

```
$ alias k=kubectl
$ complete -o default -F __start_kubectl k
```

Show client and server versions

```
$ k version
```

:config — 7 cmds

Clusters, contexts and the active namespace.

Show the merged kubeconfig

```
$ k config view
```

List contexts

```
$ k config get-contexts
```

Show the current context

```
$ k config current-context
```

Switch to another context

```
$ k config use-context my-cluster
```

Set the default namespace

```
$ k config set-context --current --namespace=my-ns
```

Create a context for a namespace

```
$ k config set-context dev --cluster=my-cluster --user=me
--namespace=dev
```

Show cluster endpoints

```
$ k cluster-info
```

:get — 14 cmds

List and inspect resources.

List pods in the current namespace

```
$ k get pods
```

List pods in every namespace

```
$ k get pods -A
```

Show node and IP for each pod

```
$ k get pods -o wide
```

Watch pods change live

```
$ k get pods -w
```

List pods by label

```
$ k get pods -l app=web
```

List services, endpoints and ingresses

```
$ k get svc,endpoints,ingress
```

List the common resources

```
$ k get all
```

Print a resource as YAML or JSON

```
$ k get pod my-pod -o yaml   $ k get pod my-pod -o json
```

Print one field with JSONPath

```
$ k get pods -o jsonpath='{.items[*].metadata.name}'
```

Print chosen columns

```
$ k get pods -o custom-
columns=NAME:.metadata.name,NODE:.spec.nodeName
```

Sort pods by age

```
$ k get pods --sort-by=.metadata.creationTimestamp
```

List only running pods

```
$ k get pods --field-selector=status.phase=Running
```

List every resource type

```
$ k api-resources
```

Explain a field of a resource

```
$ k explain pod.spec.containers
```

:pods

17 cmds

Logs, shells, port forwards and debugging.

Describe a pod and its events

```
$ k describe pod my-pod
```

Print pod logs

```
$ k logs my-pod
```

Stream pod logs

```
$ k logs -f my-pod
```

Logs from the last hour, last 100 lines

```
$ k logs my-pod --since=1h --tail=100
```

Logs from one container

```
$ k logs my-pod -c app
```

Logs from the previous crash

```
$ k logs my-pod --previous
```

Logs from every pod with a label

```
$ k logs -l app=web --all-containers
```

Open a shell in a pod

```
$ k exec -it my-pod -- sh
```

Forward a local port to a pod

```
$ k port-forward pod/my-pod 8080:80
```

Forward a local port to a service

```
$ k port-forward svc/web 8080:80
```

Copy a file out of a pod

```
$ k cp my-pod:/tmp/dump.sql ./dump.sql
```

Start a throwaway debug pod

```
$ k run debug --image=busybox -it --rm -- sh
```

Debug a running pod with an ephemeral container

```
$ k debug -it my-pod --image=busybox --target=app
```

Wait until a pod is ready

```
$ k wait --for=condition=Ready pod/my-pod --timeout=60s
```

Show pod CPU and memory (needs metrics-server)

```
$ k top pod
```

Delete a pod

```
$ k delete pod my-pod
```

Force delete a stuck pod

```
$ k delete pod my-pod --grace-period=0 --force
```

:deploy

10 cmds

Scale, update and roll back workloads.

Create a deployment

```
$ k create deployment web --image=nginx
```

Scale a deployment

```
$ k scale deploy/web --replicas=3
```

Update the container image

```
$ k set image deploy/web nginx=nginx:1.27
```

Watch a rollout finish

```
$ k rollout status deploy/web
```

Show rollout history

```
$ k rollout history deploy/web
```

Roll back to the previous version

```
$ k rollout undo deploy/web
```

Roll back to a specific revision

```
$ k rollout undo deploy/web --to-revision=2
```

Restart every pod in a deployment

```
$ k rollout restart deploy/web
```

Autoscale on CPU

```
$ k autoscale deploy/web --min=2 --max=10 --cpu-percent=80
```

Expose a deployment as a service

```
$ k expose deploy/web --port=80 --target-port=80
```

:jobs

5 cmds

One-off and scheduled work.

Run a one-off job

```
$ k create job hello --image=busybox -- echo hello
```

Create a CronJob

```
$ k create cronjob nightly --image=busybox --schedule='0 3 * * *' -- echo hi
```

Run a CronJob now

```
$ k create job nightly-manual --from=cronjob/nightly
```

List jobs and CronJobs

```
$ k get jobs,cronjobs
```

Wait for a job to finish

```
$ k wait --for=condition=complete job/hello --timeout=120s
```

:apply

9 cmds

Create and change resources from manifests.

Apply a manifest

```
$ k apply -f app.yaml
```

Apply a kustomize directory

```
$ k apply -k ./overlays/prod
```

Preview changes before applying

```
$ k diff -f app.yaml
```

Delete what a manifest created

```
$ k delete -f app.yaml
```

Edit a live resource

```
$ k edit deploy/web
```

Generate YAML without creating

```
$ k create deploy web --image=nginx --dry-run=client -o yaml > web.yaml
```

Add a label

```
$ k label pod my-pod env=prod
```

Add an annotation

```
$ k annotate deploy/web owner=team-a
```

Patch a field

```
$ k patch deploy/web -p '{"spec":{"replicas":2}}'
```

:rbac

5 cmds

Check and grant permissions.

Check if you can do something

```
$ k auth can-i create deployments
```

List everything you can do

```
$ k auth can-i --list
```

Check as a service account

```
$ k auth can-i get pods --as=system:serviceaccount:dev:app
```

Create a role

```
$ k create role pod-reader --verb=get,list,watch --resource=pods
```

Bind a role to a user

```
$ k create rolebinding read-pods --role=pod-reader --user=jane
```

:nodes

7 cmds

Cluster capacity and maintenance.

List nodes

```
$ k get nodes
```

Show node CPU and memory (needs metrics-server)

```
$ k top node
```

Stop scheduling on a node

```
$ k cordon node-1
```

Evict pods before maintenance

```
$ k drain node-1 --ignore-daemonsets
```

Allow scheduling again

```
$ k uncordon node-1
```

Taint a node

```
$ k taint nodes node-1 dedicated=gpu:NoSchedule
```

Recent events, newest last

```
$ k get events --sort-by=.metadata.creationTimestamp
```

:secrets

5 cmds

Configuration and isolation.

Create a secret from a value

```
$ k create secret generic db --from-literal=password=s3cret
```

Read and decode a secret

```
$ k get secret db -o jsonpath='{.data.password}' | base64 -d
```

Create a ConfigMap from a file

```
$ k create configmap app --from-file=config.yaml
```

Create a namespace

```
$ k create namespace my-ns
```

List namespaces

```
$ k get ns
```

:short 14 cmds

Type less. Every resource has a short name.

pods

`po`

services

`svc`

deployments

`deploy`

replicasets

`rs`

statefulsets

`sts`

daemonsets

`ds`

cronjobs

`cj`

namespaces

`ns`

nodes

`no`

configmaps

`cm`

persistentvolumeclaims

`pvc`

ingresses

`ing`

horizontalpodautoscalers

`hpa`

serviceaccounts

`sa`

Updated Nov 8, 2026 · checked against kubectl 1.30+ · sources: kubectl quick reference, kubectl command reference

Made by Lucas Bittencourt. Free, no ads, no tracking. Spotted a mistake? Tell me.